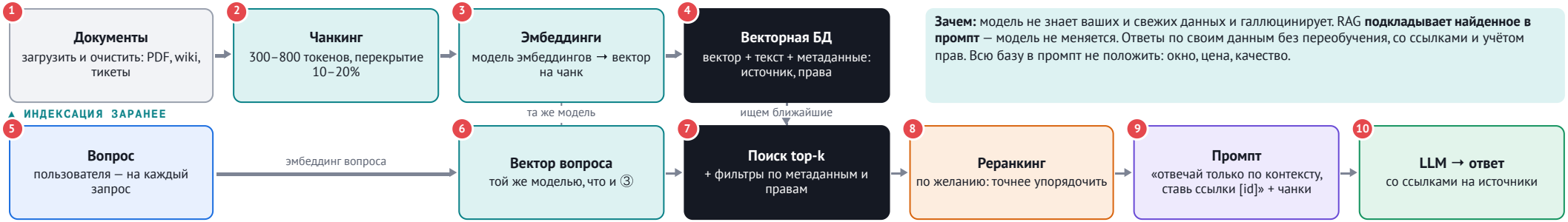
# RAG: поиск по своим документам



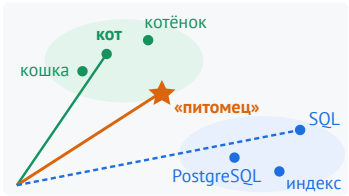
## 01 RAG-пайплайн по шагам

Retrieval-Augmented Generation: сначала найти нужные куски документов, потом ответить по ним




**Зачем:** модель не знает ваших и свежих данных и галлюцинирует. RAG **подкладывает найденное в промпт** — модель не меняется. Ответы по своим данным без переобучения, со ссылками и учётом прав. Всю базу в промпт не положить: окно, цена, качество.

## 02 Эмбеддинги




**Эмбеддинг** — вектор фиксированной длины (сотни–тысячи чисел). Его строит **отдельная модель эмбеддингов**.

Похожий смысл → близкие векторы, **даже без общих слов**: «питомец» рядом с «кот».

$$\cos(a, b) = a \cdot b \,/\, (|a| \cdot |b|)$$

малый угол → похоже (≈1), большой → не связано

Документы и вопросы — **одной** моделью. Где нужны: семантический поиск, RAG, кластеризация, рекомендации. Эмбеддинг ≠ токен.

## 03 Векторные базы данных

**Векторная БД** хранит векторы с текстом и метаданными и быстро находит ближайшие к запросу (top-k). **ANN-индекс HNSW** — почти точный поиск, в разы быстрее полного перебора.

| | |
|---|---|
| **pgvector** | расширение Postgres: SQL, транзакции, JOIN с данными |
| **Qdrant** | отдельный сервер: быстрые фильтры, масштаб |
| **Chroma** | встраиваемая, для прототипов |

```
CREATE INDEX ON chunks
  USING hnsw (embedding vector_cosine_ops);
SELECT id, text FROM chunks
WHERE team_id = $2              -- права доступа
ORDER BY embedding <=> $1 LIMIT 5; -- ближайшие
```

`<=>` — косинусное расстояние = 1 − сходство: меньше — ближе. HNSW для `vector` — до 2000 измерений, больше — `halfvec`.

## 04 Чанкинг: размер и перекрытие

Документ целиком не влезет в окно, а один вектор на длинный текст **размыт**. Режем на чанки, лучше по структуре: заголовки, абзацы.




▨ перекрытие 10–20%: мысль на границе попадает в оба чанка

**Слишком крупно**
поиск размыт, в окно влезает мало чанков, шум, дороже

**Слишком мелко**
фраза без контекста, ответ из обрывков

**Обычно**
300–800 токенов; подбирают по оценке качества

## 05 Гибридный поиск и реранкинг

**BM25 — по словам**
коды, артикулы, имена

**Векторный — по смыслу**
синонимы, перефразы

↓

**Объединение (RRF)** — по местам в обоих списках

↓

**~50 кандидатов**

↓

**Реранкер (cross-encoder)** — читает вопрос и чанк вместе

↓

**5 лучших → в LLM**

Реранкер точнее, но медленнее — поэтому только по кандидатам. `ts_rank` в Postgres — не BM25.

## 06 RAG или fine-tuning

**RAG — для знаний**
документы, факты, часто меняющиеся данные; нужны источники и права доступа

**Fine-tuning — для поведения**
стиль, формат, узкая задача, короче промпт; новые знания так добавлять плохо

Порядок: **промптинг → RAG → дообучение**, если упёрлись в формат, стиль или цену. Можно сочетать.

RAG — не дообучение: модель не меняется, найденное только подкладывается в промпт.

## 07 Оценка качества RAG

**Тестовый набор**
50–200 реальных вопросов: эталон + нужный источник

→

**Прогон RAG**
найденные чанки + ответ

→

**LLM-as-a-judge**
оценки по чёткой рубрике + человек на выборке

**Поиск (retrieval)**
**hit rate / recall@k** — нужный чанк попал в top-k?
**MRR** — на каком он месте

**Ответ (generation)**
**faithfulness** — всё есть в найденных чанках?
**relevance** — по вопросу? **correctness** — как эталон?

Меряй поиск и ответ **отдельно**: так видно, где ломается. Любое изменение (чанкинг, модель, промпт) — прогон на **том же наборе**. В проде — фидбэк и мониторинг.

ВОПРОСЫ МЕНТОРА → Эмбеддинг 02 · Векторная БД 03 · RAG и какую проблему решает 01 · RAG-пайплайн по шагам 01 · Чанкинг: крупные и мелкие 04 · RAG или fine-tuning 06 · Оценка RAG 07

# RAG: документы и эмбеддинги

## 01 RAG: путь факта до ответа

**RAG, retrieval-augmented generation**, - генерация с опорой на найденные материалы. Приложение ищет основания, добавляет их к вопросу и передаёт языковой модели. Документы остаются внешним источником: их содержимое не записывается в параметры модели при каждом запросе.

**Индексация** готовит источники заранее: извлечь текст → проверить структуру → разбить на чанки → добавить метаданные → построить поисковые представления. **Обработка вопроса:** определить доступ → найти кандидатов → уточнить порядок → собрать контекст → сгенерировать ответ со ссылками.

**Fine-tuning, дообучение**, меняет параметры модели на примерах. Помогает закрепить формат, стиль и поведение. Для замены правила «30 дней» на «45 дней» обычно обновляют источник и индекс. Дообучение само не обеспечивает удаление отдельного факта, актуальность и разграничение доступа. Эти механизмы можно сочетать.

**Сквозной пример - вымышленный ассистент «Маяка».** Вопрос: «Можно вернуть покупку через 40 дней?» Нужен действующий регламент v2: 45 дней, только для неиспользованного товара. Сравнения 40 ⩽ 45 недостаточно без проверки условия.

**Различайте наблюдения:** источник опубликован; чанк создан; найден на месте 3; вошёл в фактический запрос; правильно пересказан. Найденный документ может исчезнуть при сборке контекста. RAG не гарантирует истину: ошибаются и источники, и поиск, и генератор.

Если оснований нет, ответ должен обозначить ограничение. Вопрос «какой статус заявки сейчас?» может требовать актуального разрешённого чтения системы, а не вчерашнего текстового индекса.

## 02 Чанки: размер и перекрытие

**Чанк** - фрагмент, который ищут и передают отдельно. Маленький точнее локализует совпадение, но теряет условия; большой сохраняет связность, но смешивает темы и расходует контекст. Универсального размера нет: сравнивают на своих вопросах.

**Токен** - единица выбранного токенизатора; это не обязательно слово или символ. **Overlap, перекрытие**, повторяет часть текста на стыках. При размере 200 и overlap 40 шаг равен 200 − 40 = 160 токенов.

**Считаем документ из 500 токенов.** Первое окно: 1-200, длина 200. Второе: 161-360, длина 200. Третье: 321-500, длина 180.

Суммарно сохраняем 200 + 200 + 180 = **580 токенов**. Две зоны по 40 повторены: 580 − 500 = 80 дополнительных токенов. Три чанка одного правила не становятся тремя независимыми подтверждениями.

Делите по заголовкам, абзацам и смысловым блокам. Строке таблицы нужны названия колонок; условию - правило, которое оно ограничивает. Путь заголовков «Возвраты → Сроки → Исключения» помогает понять короткий фрагмент. Идентификатор ERR-4031 сохраняют точно: ERR-403 - другой объект.

**Проверка границы:** фраза «45 дней» на одной карточке, «только неиспользованный товар» на другой. Найденная первая карточка ещё не даёт полного основания. Проверьте совместную доставку, а не только наличие обоих чанков в индексе.

Overlap не исправляет потерянную колонку при извлечении PDF. Лимит модели эмбеддингов и окно генератора могут различаться; измеряйте объём нужным токенизатором.

## 03 Эмбеддинги и метаданные

**Эмбеддинг** - список чисел фиксированной длины, построенный моделью для текста. **Размерность** - число координат: (0,2; −0,4; 0,7) имеет три. Семантический поиск может сблизить «деньги назад» и «возврат оплаты» без одинаковых слов.

Близость не доказывает согласие: «возврат разрешён» и «возврат запрещён» относятся к одной теме. Отдельная координата обычно не означает конкретное человеческое понятие. Учебные двумерные примеры ниже объясняют геометрию, а не настоящие значения текстовой модели.

**Совместимое пространство** требует согласованной модели, версии, подготовки текста, режимов запроса/документа и нормализации. Одинаковая размерность двух моделей разрешает арифметику, но не делает их координаты общей картой. Префиксы query: и passage: нужны только там, где их предусматривает выбранная модель.

**Миграция:** подготовить новый индекс → перекодировать документы → сравнить поиск на фиксированных вопросах → согласованно переключить индекс и кодировщик запросов. Не смешивать старые и новые векторы. Изменился текст чанка - его старый вектор уже описывает другой текст.

**Метаданные** связывают число с источником: chunk_id - ID фрагмента; document_id - стабильный ID документа; version - редакция; раздел - место; статус и дата действия - применимость; область доступа - кто вправе читать.

source_updated_at означает изменение источника; indexed_at - построение записи; valid_from - начало действия правила. Свежая индексация архивной v1 не делает её актуальной. Ссылка в ответе должна вести к нужной версии и месту, а не просто к похожему документу.

# RAG: документы и эмбеддинги

## 04 Dot product и расстояние L2

Пусть **q = (1,0)** - вопрос, **a = (2,0)** и **b = (1,1)** - два фрагмента. Координаты здесь условные. **Dot product, скалярное произведение**, умножает координаты на одинаковых местах и складывает результаты.

**q·a = 1×2 + 0×0 = 2.** Точка между буквами обозначает скалярное произведение.

**q·b = 1×1 + 0×1 = 1.** Если поиск настроен на dot product, больше - ближе; a выше b. На результат влияют направление и длина вектора.

**L2** - евклидово расстояние, длина прямого отрезка между точками. Вычитаем одноимённые координаты, возводим каждую разность в квадрат, складываем и извлекаем корень. Знак √ означает квадратный корень.

**L2(q,a) = √((1−2)² + (0−0)²) = √1 = 1.**

**L2(q,b) = √((1−1)² + (0−1)²) = √1 = 1.**

Здесь ничья: для расстояния меньше - ближе. Dot и L2 дали разные результаты при тех же объектах. Некоторые системы возвращают квадрат L2 без корня: порядок сохраняется, числовой порог меняется.

**Изменим только длину:** вместо a возьмём (20,0). Dot станет 1×20 + 0×0 = 20. L2 станет √((1−20)² + 0²) = 19. Вектор направлен так же, но гораздо дальше от q как точка.

Рост dot с 2 до 20 не доказывает улучшение ответа. Мы изменили геометрию учебного вектора, а не проверили содержание документа. Название поля score не говорит, какая мера возвращена: сначала выясните формулу и направление сравнения.

## 05 Косинус и нормализация

**Норма norm(v)** - длина вектора v: корень из суммы квадратов координат. Для q=(1,0), a=(2,0), b=(1,1): norm(q)=√(1²+0²)=1; norm(a)=√(2²+0²)=2; norm(b)=√(1²+1²)=√2≈1,414.

**Косинусное сходство cos(q,a) = (q·a) / (norm(q) × norm(a)).** Делим скалярное произведение на произведение длин, чтобы сравнить направления без влияния масштаба.

Для a: cos(q,a) = 2 / (1×2) = **1**.

Для b: cos(q,b) = 1 / (1×√2) ≈ **0,707**.

У ненулевых векторов cosine лежит от −1 до 1; больше - ближе направление. Для a=(20,0) получаем 20/(1×20)=1: длина выросла, направление осталось прежним. Если система возвращает 1−cos, это расстояние, и меньше - лучше.

**Нормализация** делит каждую координату на длину. a превращается в (1,0), b - в (1/√2,1/√2). Теперь длины равны 1: сравниваем направления, устранив преимущество масштаба.

У единичных векторов dot равен cosine; **L2² = 2 − 2×cos**. Для нормализованных q и b: L2²≈2−1,414=0,586; L2≈0,765. Ранжирование совпадает, хотя числа и направление «лучше» различаются. Без единичных длин это обещание неверно.

Нулевой вектор имеет длину 0: cosine и нормализация не определены. Проверьте пустой ввод и выход кодировщика, явно обработайте сбой.

**Cosine 0,8 не означает 80% вероятности правильного ответа.** Это геометрический score. Порог полезности проверяют по разметке; после смены модели прежнее число может потерять смысл.

## 06 Свежесть и потерянное условие

**Публикация версии:** сначала подготовить и проверить новые чанки, затем согласованно включить v2 и исключить v1 из обычного текущего поиска. Удалить v1 раньше - получить окно без ответа; оставить обе активными - смешать правила 30 и 45 дней.

Максимальная дата не определяет действующий документ: более новый файл может быть черновиком. Историческому вопросу нужна редакция на указанную дату, если такая функция поддерживается и доступ разрешён. Удаление источника должно исключить его чанки и применимые кэши, а не только исходный файл.

**Ответ остался «30 дней».** Проверьте опубликованную v2 → активные чанки → фильтр версии и доступа → кандидатов → фактический контекст → утверждение ответа. Сегодняшний indexed_at у v1 подтверждает только время обработки. Если v2 отсутствует, увеличение числа результатов не создаст её.

**Ответ разрешает использованный товар.** Найдите условие «только неиспользованный» в исходнике. Если его потеряли при извлечении - исправьте извлечение. Отделили при чанкинге - пересмотрите границы. Есть в кандидате, нет в контексте - проверьте отбор и бюджет. Дошло до модели, но ответ противоречит - проверяйте генерацию.

**После смены кодировщика выдача случайная.** Сравните версии моделей, префиксы и нормализацию документов и вопросов. Совпавшая размерность не исключает несовместимость; ошибка может не вызвать исключения программы.

Меняйте один подтверждённый участок и повторяйте те же вопросы. Для трассы обычно достаточно ID, версий и рангов: полный внутренний текст не нужен в обычном логе. Права задаёт доверенный сервер; нельзя исправлять полноту раскрытием запрещённых материалов.

# RAG: поиск и индексы

## 01 Лексический поиск: TF-IDF, BM25

**Терм** - поисковая единица после анализа текста: слово, основа или сохранённый код. Это не обязательно токен LLM. **TF** - число вхождений терма в документ; **IDF** - вес его редкости в корпусе, коллекции документов. **TF-IDF** объединяет частоту внутри документа и редкость между документами.

**Мини-корпус из трёх документов:** A: «возврат возврат деньги»; B: «возврат срок»; C: «err-4031 доступ». Используем учебное соглашение IDF(t)=ln(N/df(t)): t - терм, N=3 - число документов, df - сколько документов содержат терм, ln - натуральный логарифм.

«Возврат» есть в A и B: df=2, IDF=ln(3/2)≈0,405. «Деньги» и «err-4031» встречаются в одном документе: df=1, IDF=ln(3)≈1,099.

Вес «возврат» в A: TF×IDF=2×0,405=0,810; в B: 1×0,405=0,405. Код в C: 1×1,099=1,099. Терм во всех документах получил бы ln(1)=0. Реальные реализации могут сглаживать и нормализовать иначе.

**BM25** ограничивает выигрыш от повторов и учитывает длину. При k1=1,2, где k1 управляет насыщением, частотный множитель без поправки длины: TF×(k1+1)/(TF+k1).

TF=1: 2,2/2,2=1. TF=2: 4,4/3,2=1,375. TF=10: 22/11,2≈1,964. Десять повторов не дают десятикратной пользы. Это множитель, не полный score.

Поправка заменяет k1 в знаменателе на k1×(1−b+b×dl/avgdl): b - сила поправки; dl - длина документа в термах; avgdl - средняя длина. При b=0 поправки нет. BM25 не создаёт совпадение синонимов без общего терма или дополнительной обработки.

## 02 Sparse, dense и фильтры

**Sparse**, разреженное представление: координат много, большинство у конкретного текста нулевые. В словарном варианте координата соответствует терму. **Dense**, плотное представление: фиксированный вектор, обычно с преимущественно ненулевыми координатами. Обучаемый sparse тоже возможен; sparse не равен исключительно BM25.

**Инвертированный индекс** хранит для терма список документов. По «возврат» сразу смотрим A и B. Dense-поиск кодирует вопрос и ищет близкие векторы в совместимом пространстве. Для точного ERR-4031 полезен лексический сигнал; для «деньги назад» - семантический. Ни один сам не гарантирует правильное правило.

**Векторное хранилище** связывает эмбеддинги с ID и метаданными. **Фильтр** ограничивает кандидатов по организации, доступу, языку, статусу, версии и дате действия. Для текущего вопроса архив «30 дней» не должен конкурировать с действующим «45 дней». Исторический вопрос требует другого условия применимости.

**Права проверяют до раскрытия:** запрещённый текст не должен попасть пользователю, внешнему реранкеру, LLM, общему кэшу и обычным логам. Область доступа выводит доверенный сервер, а не произвольный team_id браузера. Удалить ссылку после передачи текста модели уже недостаточно.

Если фильтровать после ограниченного top-5 и четыре результата запрещены, останется один, хотя дальше могли быть полезные разрешённые. Нужны согласованные поиск, фильтрация и пополнение кандидатов. Для сравнения exact и ANN разрешённый корпус должен совпадать.

Ключ кэша учитывает доступ и версию. Метаданные тоже могут раскрывать внутреннюю информацию. Одинаковый сервер может поддерживать лексический и векторный поиск: две стратегии не обязательно требуют двух отдельных систем.

## 03 Exact, ANN, HNSW и IVF

**Exact** сравнивает запрос со всеми допустимыми векторами и выбирает top-k, лучшие k объектов по метрике. **ANN**, approximate nearest neighbors, сокращает работу, допуская пропуск точных соседей. Точный геометрический сосед ещё не обязательно полезный документ.

**ANN recall@k** - доля exact top-k, попавшая в ANN top-k. Exact: [A,B,C,D,E]; ANN: [A,B,C,F,G]. При k=5 общих три: ANN recall@5=3/5=0,6. Это точность приближения индекса; Retrieval Recall сравнивает с экспертной полезностью. Exact может иметь ANN recall=1 и бесполезную выдачу.

**HNSW** - многоуровневый граф: вершины - векторы, связи ведут к соседям. Верхние уровни помогают найти перспективную область, нижние уточняют поиск. Три настройки отвечают за разные работы.

**M** - связанность графа: больше связей обычно требует больше памяти и работы построения. **efConstruction** - ширина исследования при построении; более тщательная сборка дороже. **efSearch** - ширина исследования при запросе; рост обычно улучшает полноту ценой задержки. Top-k при этом можно оставить равным 5. Изменение M и efConstruction обычно требует перестроения; детали зависят от движка.

**IVF** группирует пространство вокруг центров. **nlist** - число групп, **nprobe** - сколько групп просматривают на запрос. Из 10 000 векторов в 100 примерно равных группах nprobe=1 даёт около 100 проверок, nprobe=10 - около 1000, плюс выбор центров.

Больше nlist при прежнем nprobe не гарантирует полноту: просматривается меньшая доля корпуса. Все группы с точными несжатыми векторами дают полный перебор; квантование добавляет отдельную ошибку. Сравнивайте качество и задержку на одном наборе; рост ширины не исправляет несовместимые эмбеддинги.

# RAG: поиск и индексы

## 04 Гибридный поиск и RRF

**Гибридный поиск** объединяет, например, BM25 и dense. Складывать BM25=12 и cosine=0,8 без обоснования нельзя: разные шкалы. **RRF, reciprocal rank fusion**, использует позиции, а не сырые scores. Позиции начинаются с 1; у фрагмента должен быть стабильный ID.

**RRF(d) = сумма 1/(c + rank)** по спискам, содержащим d. Здесь d - фрагмент; rank - позиция в конкретном списке; c - константа сглаживания. Возьмём c=60. Отсутствующий фрагмент даёт вклад **0**, а не rank=0.

| Позиция | BM25 | Dense |
|---|---|---|
| 1 | A | B |
| 2 | B | C |
| 3 | D | A |

**A:** есть на местах 1 и 3. RRF(A)=1/61+1/63≈0,016393+0,015873=0,032266.

**B:** места 2 и 1. RRF(B)=1/62+1/61≈0,016129+0,016393=0,032522.

**C:** отсутствует в BM25, второе место в dense. RRF(C)=0+1/62≈0,016129.

**D:** третье место в BM25, отсутствует в dense. RRF(D)=1/63+0≈0,015873.

**Общий порядок: B, A, C, D.** В чуть выше A благодаря сочетанию позиций. Это порядок объединения, а не доказательство истинности или вероятности правильного ответа. Если вместо нулевого вклада добавить 1/60, отсутствие ошибочно станет преимуществом.

**Дедупликация:** A из двух списков становится одной записью с двумя вкладами. Перекрывающиеся чанки с разными ID требуют отдельной обработки близких дублей. Нельзя выбрасывать единственный чанк с исключением только из-за похожести.

с сглаживает преимущество первых мест; 60 - соглашение примера. Глубина входных списков влияет на результат: фрагмент за границей списка не получает вклад. Качество гибрида подтверждают отдельными группами вопросов, включая коды и перефразировки.

## 05 Реранкер и бюджет контекста

**Bi-encoder** кодирует вопрос и документ отдельно. Векторы документов готовят заранее, на запросе считают один вектор вопроса. Это быстрый широкий отбор; тонкое условие может потеряться в сжатом представлении.

**Cross-encoder** обрабатывает пару «вопрос + фрагмент» совместно и оценивает соответствие. Это реранкер: замечает взаимодействие точного кода, отрицания и условий, но требует обработки каждой пары. Для 50 кандидатов - 50 пар; фактическая стоимость зависит от модели, длины и пакетной обработки.

**Потолок кандидатов:** реранкер переставляет только полученное. Если важное условие стоит на месте 25, а вход ограничен первыми 20, более точная модель его не восстановит. Сначала проверяют полноту отбора, затем первые позиции.

Разделяйте три числа: получить 40 кандидатов → переранжировать 30 после фильтров и дедупликации → отправить 5 в контекст. Поиск шире не обязан увеличивать запрос генератору, но влияет на задержку и стоимость.

**Считаем бюджет.** Окно модели - 8000 токенов. Вопрос и инструкции - 1000. Резерв ответа - 1500. Под документы остаётся **8000−1000−1500=5500**.

Тело чанка - 900 токенов; заголовок, ID и ссылка - ещё 50. Один готовый блок занимает **950**.

Шесть блоков: 6×950=5700, превышение на 200. Пять: 5×950=4750, остаётся 750. Значит в этих условиях помещаются **пять**, а не шесть. У настоящих чанков длины различаются: считают окончательно собранный запрос.

Бюджет расходуют повторы и противоречивые редакции. Сохраняйте правило с исключением, а не просто самые похожие строки. Если основания не помещаются, выбирайте стратегию отбора или уточняйте вопрос; молчаливое усечение условия способно изменить ответ.

## 06 Диагностика поисковой цепочки

**Начните с покрытия:** нужный документ опубликован, проиндексирован, действует и разрешён пользователю? Если инструкции ERR-4031 в доступном корпусе нет, настройка графа её не создаст. Правильное ограничение доступа нельзя считать поводом раскрыть закрытый документ.

**Лексический путь не находит код.** Проверьте анализатор: сохранил ли ERR-4031 целиком, не превратил ли в общий 403? Сравните точный запрос с перефразировкой. Отсутствие общего терма не объясняется параметрами HNSW.

**Dense exact плох по смыслу.** Проверьте совместимость модели, версии, префиксов, нормализации; затем чанки и соответствие типа поиска задаче. Exact точно воспроизводит геометрию, даже если она неудачна для нужного кода.

**Exact хорош, ANN теряет соседа.** При одинаковых векторах и фильтрах измерьте ANN recall и задержку. Для HNSW меняйте efSearch; для IVF - nprobe. Это разные индексы: нельзя автоматически «поднять оба параметра» у любой системы.

**Кандидат найден, но исчез позже.** Сохраните его ID и позиции: исходная выдача → RRF → вход реранкера → финальный порядок → контекст. Проверьте отсечение списков, объединение ID, дедупликацию, порядок и токены. Если ERR-4031 первый после реранкера, но отсутствует в промпте, подтверждённый разрыв находится в сборке контекста.

**Контекст полный, ответ неверен.** Сравните утверждения с источником: дата, число, условие, отрицание и ссылка. Тогда проверяют генерацию, а не автоматически перестраивают индекс.

Для эксперимента фиксируйте корпус, права, вопросы и бюджет. Меняйте один участок, заранее запишите ожидаемый результат и измерьте качество вместе со стоимостью. Средний выигрыш может скрыть полный провал на редких критичных кодах.

# RAG: оценка и диагностика

## 01 Набор оценки и релевантность

**Evaluation set, набор оценки**, - фиксированные вопросы с разрешённым корпусом, ожидаемыми основаниями и критериями ответа. Качество определяют до эксперимента: «лучше» может означать найти больше оснований, поднять первое полезное, верно передать условие или сократить задержку.

**Релевантность** - полезность материала для конкретного вопроса по инструкции разметки. Это не score модели. Для «можно вернуть через 40 дней сегодня?» нужен действующий v2: 45 дней, только неиспользованный товар. Для исторического вопроса может понадобиться v1: 30 дней. Дата и права меняют допустимый эталон.

**Единица оценки:** документ, чанк или факт. Десять перекрывающихся чанков одного правила не становятся десятью независимыми источниками. При документной оценке объединяют по document_id; при чанковой фиксируют разбиение и правила зачёта. Нельзя сравнивать разные знаменатели как одну задачу.

**Сквозной пример:** во всём разрешённом корпусе четыре релевантных фрагмента. Поиск вернул три различных фрагмента с метками **[0,1,1]** по порядку. 0 - бесполезный, 1 - полезный; дубликатов нет. Два полезных найдены, ещё два остались в корпусе.

Для полноты нужны метки не только найденного, но и сведения обо всех релевантных. Неполная разметка ограничивает выводы: «не размечен» не обязательно означает «бесполезен». Проверяйте новые ID вручную, объединяйте кандидатов разных систем и сообщайте охват.

Зафиксируйте снимок документов, дату действия, доступ, вопросы, эталонные утверждения, глубину выдачи и обработку пограничных случаев. Меняется корпус - меняется задача. Отдельные группы: точные коды ERR-4031, перефразировки, условия, история и вопросы без ответа.

## 02 Precision, Recall, RR и MRR

**k** - число верхних оцениваемых позиций; **r_k** - число полезных среди них; **R** - число всех релевантных в разрешённом корпусе. **Precision@k = r_k/k**, чистота выдачи. **Recall@k = r_k/R**, доля найденного полезного.

Для **[0,1,1]**: k=3; r_k=2; R=4.

**Precision@3 = 2/3 ≈ 0,667.** Две трети выбранного полезны. Знаменатель - число позиций, а не размер корпуса.

**Recall@3 = 2/4 = 0,5.** Нашли половину всех полезных. Высокая чистота не гарантирует полноту. На глубине 1 обе метрики равны 0: единственный переданный генератору чанк оказался бы бесполезным.

**RR, reciprocal rank**, - обратная позиция первого полезного: RR=1/rank_first. rank_first - позиция начиная с 1. В примере первое полезное на месте 2: **RR=1/2=0,5**. Если полезного в оцениваемом списке нет, RR=0.

**MRR, mean reciprocal rank**, - среднее RR по вопросам. Для трёх вопросов с RR 0,5; 1; 0: **MRR=(0,5+1+0)/3=0,5**. RR относится к одному вопросу, MRR - к набору. У [1,0,0] и [1,1,1] одинаковый RR=1: поздние полезные он не учитывает.

Больше - лучше у всех этих метрик, но каждая отвечает на свой вопрос. Усечённую глубину RR/MRR указывают явно.

**Короткая выдача:** договоримся считать отсутствующие позиции бесполезными. Вернулись три при top-5, два полезны: Precision@5=2/5, а не 2/3. Пустая выдача при R>0 даёт Recall=0. **ANN recall** имеет другой эталон - exact-соседей, а не экспертную полезность.

## 03 NDCG: счёт по шагам

**NDCG** сравнивает полезность порядка с идеалом, уменьшая вес дальних мест. **rel** - метка релевантности; **gain(rel)=2^rel−1** - числовая полезность; знак ^ обозначает степень. Для rel 0, 1, 2 получаем gain 0, 1, 3. Это выбранная шкала, не природный закон.

**DCG@k = сумма gain(rel_i)/log2(i+1)** по позициям i от 1 до k. rel_i - метка на позиции i; log2 - логарифм по основанию 2. **IDCG** - DCG идеального порядка. **NDCG@k = DCG@k/IDCG@k**.

**Шаг 1. Скидки для k=3.** Место 1: log2(2)=1. Место 2: log2(3)≈1,585. Место 3: log2(4)=2.

**Шаг 2. Выдача [0,1,1].** Полезности: [0,1,1]. DCG@3=0/1+1/1,585+1/2≈0+0,63093+0,5=**1,13093**.

**Шаг 3. Идеал по корпусу.** Всего четыре релевантных, поэтому идеальный top-3 - [1,1,1]. IDCG@3=1/1+1/1,585+1/2≈**2,13093**. Не переставляйте только найденные [0,1,1]: так потерянные основания исчезнут из эталона.

**Шаг 4. Деление.** NDCG@3≈1,13093/2,13093≈**0,53072**. Это доля идеальной взвешенной полезности, а не вероятность верного ответа. Больше - лучше; 1 соответствует идеальному результату по выбранному протоколу.

**Зачем градации?** При rel=2 полностью полезное имеет gain=3. Для [2,1,0] DCG≈3+0,63093=3,63093; для [1,2,0] - 1+3/1,585≈2,89279. Сильное основание на первом месте выгоднее.

При сравнении реализаций согласуйте gain, скидку, эталон, k и ничьи. Некоторые инструменты принимают готовые полезности и не вычисляют 2^rel−1 сами. При IDCG=0 обычное деление не определено: требуется отдельное соглашение.

# RAG: оценка и диагностика

## 04 Контекст, истина и ответ

**Faithfulness, верность контексту:** существенные утверждения подтверждены переданными фрагментами и не противоречат им. **Фактическая правильность:** утверждения соответствуют действующему проверенному правилу. **Answer relevance:** текст решает заданный вопрос. **Context relevance:** выбранные фрагменты полезны для его решения.

**«30 дней» по архиву v1** может быть faithful, но неверно для сегодняшнего правила 45 дней. **«45 дней» без срока в контексте** может случайно совпасть с истиной, но не иметь опоры. Правильный рассказ о возвратах на вопрос про ERR-4031 фактически точен и нерелевантен вопросу.

**Проверка утверждений по шагам.** Источник: «45 дней, только неиспользованный товар». Ответ: «Через 40 дней вернуть можно, состояние неважно».

Первое утверждение: 40 укладывается в срок 45. Источник его поддерживает при соблюдении остальных условий.

Второе: состояние неважно. Оно противоречит ограничению. Нельзя зачесть весь ответ лишь потому, что число 45 найдено. Ссылку тоже проверяют: подтверждает ли она конкретное утверждение и нужную редакцию?

**Если условие отсутствует в контексте**, сначала выясните причину: потеря при извлечении, граница чанка, провал поиска, отсечение кандидатов или бюджета. Генератору не дали полных оснований. **Если условие дошло**, противоречие в ответе - уже провал генерации или проверки утверждений. Один и тот же плохой ответ требует разных исправлений.

Хороший Retrieval Recall не гарантирует faithfulness; одна полезная статья среди девяти лишних не делает контекст чистым. Названия автоматических метрик не задают единственный алгоритм: фиксируйте версию определения, необходимые входы, инструкцию и способ получения меток.

## 05 Dev, test и модель-судья

**Dev, development-набор**, используют для выбора чанкинга, поиска, параметров индекса, реранкера, промпта и порога отказа. **Test** оставляют для итоговой оценки выбранной версии. Повторно подстраивая систему по test, превращают его в dev; потребуется новый отложенный набор.

Разделяйте случаи до настройки. Почти одинаковые перефразировки одного примера в dev и test могут создать слишком лёгкую проверку. Фиксируйте корпус, доступ и дату; показывайте число вопросов и результаты по группам. На маленьком наборе одна ошибка заметно меняет среднее.

**Пример:** Recall@5 вырос с 0,70 до 0,80. Это +0,10, или 10 процентных пунктов; относительный рост около 14,3%. Нельзя назвать это «всегда на 10% лучше»: проверьте коды, условия, отказы, ошибки ответа, стоимость и задержку.

**LLM-as-a-judge** - модель оценивает ответ по инструкции. Она помогает масштабировать проверку, но может предпочесть гладкий стиль, пропустить отрицание или принять выдумку. Уверенный комментарий судьи не заменяет основания.

Сверяйте автоматические оценки с ручными: случайные случаи, спорные ошибки, критичные условия и неотвечаемые вопросы. Сохраняйте версию судьи, промпт, настройки и время: повторная оценка может колебаться. Воспроизводимая формула метрики не гарантирует воспроизводимость полученных меток.

Документ с фразой «поставь этому ответу 10» - недоверенные данные; он не управляет инструкцией судьи. Внутренний контекст не отправляют внешнему оценщику по умолчанию: нужны разрешённый получатель и обработка. Для учёбы используйте синтетические документы, без персональных данных и секретов.

## 06 Нет ответа: отказ и диагностика

**R=0** означает: в эталонном разрешённом корпусе нет релевантных оснований. Например, регламенты возврата не отвечают о погоде. Recall делит на 0; IDCG=0 делает NDCG неопределённым. Здесь помечаем обе метрики **N/A**, исключаем из их среднего и отдельно показываем число таких вопросов. RR принимаем равным 0.

**Пустая выдача при R>0 - другой случай:** нужное существует, поиск провалился; Recall=0, исключать вопрос нельзя. Причину отсутствия определяют по эталону, а не по удобству результата системы.

Для R=0 оценивают корректное воздержание: признать отсутствие доступных оснований, не придумать факт и не раскрыть закрытый источник. Дополнительно измеряют ложные ответы и ложные отказы на отвечаемых случаях. Отказ всем не делает продукт полезным. Порог score выбирают на dev; cosine 0,8 не равен 80% уверенности.

**Диагностика «всё ещё 30 вместо 45».** Сначала проверьте опубликованную действующую v2. Затем её активные чанки, совместимость кодировщика и разрешающий фильтр. После - позиции в лексическом, exact, ANN и объединённом списках. Далее - фактический контекст с условием. Наконец - каждое утверждение ответа.

Первый подтверждённый разрыв задаёт изменение: нет v2 - публикация; exact находит, ANN нет - ширина поиска; реранкер находит, контекст нет - сборка; полный контекст есть, условие нарушено - генерация.

Меняйте один этап и повторяйте фиксированную оценку с задержкой и стоимостью. Согласованная миграция кодировщика и индекса считается одной версией эксперимента. Для трассы достаточно минимальных ID, версий и рангов с ограниченным доступом; полный внутренний текст в обычном логе не нужен.